

ArtisanHub — Guide détaillé de déploiement sur Render

Document technique | Laravel 12, PHP 8.2, Docker, PostgreSQL, worker et scheduler Render

Sécurité : ne jamais mettre une clé API, un mot de passe, un fichier `.env` ou une clé privée dans GitHub. Les secrets doivent être saisis uniquement dans Render.

1. Pré-requis

- Un compte GitHub contenant le projet ArtisanHub.
- Un compte Render : dashboard.render.com.
- Une branche main contenant `render.yaml`, `Dockerfile` et `deploy/docker-entrypoint.sh`.
- Les accès Fedapay, SMTP et S3/R2 si ces services sont utilisés.
- Une clé d'application Laravel générée avec PHP localement.

2. Préparer le dépôt Git

```
cd /home/landry/artisanhub.worktrees/greeting-in-french

git status

git remote add origin https://github.com/TON_COMPTE/TON_REPO.git

git push -u origin main
```

Si le remote existe déjà :

```
git remote -v

git push -u origin main
```

Vérifier que `.env`, `vendor`, `node_modules` et `public/build` ne sont pas suivis :

```
git check-ignore -v .env vendor node_modules public/build
```

3. Créer le Blueprint Render

1. Ouvrir [Render Dashboard](#).
2. Cliquer sur **New**, puis **Blueprint**.
3. Connecter GitHub et sélectionner le dépôt ArtisanHub.
4. Sélectionner la branche main.
5. Render détecte le fichier `render.yaml`.
6. Vérifier les services proposés : `artisanhub-web`, `artisanhub-worker`, `artisan-scheduler` et `artisanhub-db`.
7. Cliquer sur **Apply**.

Le web service utilise le `Dockerfile`. Le worker exécute la queue Laravel et le cron exécute le scheduler chaque minute.

4. Générer la clé APP_KEY

```
cd /home/landry/artisanhub.worktrees/greeting-in-french  
  
php artisan key:generate --show
```

Copier toute la valeur affichée, généralement sous la forme base64 : . . . Ne pas utiliser une clé différente pour le worker.

5. Configurer les variables Render

Dans le service `artisanhub-web` : **Environment** → **Add Environment Variable**.

Variable	Valeur à mettre	Remarque
APP_ENV	production	Valeur fixe.
APP_DEBUG	false	Ne jamais activer en production.
APP_KEY	base64:...	Clé générée avec Artisan.
APP_URL	https://artisanhub-web.onrender.com	Remplacer par l'URL Render réelle.
APP_LOCALE	fr	Langue principale.
APP_FALLBACK_LO	fr	Langue de secours.

CALE

LOG_CHANNEL	stderr	Permet de lire les logs dans Render.
LOG_LEVEL	warning	Réduire le bruit en production.
DB_CONNECTION	pgsql	La base Render est PostgreSQL.
QUEUE_CONNECTION	database	Compatible avec la table jobs.
SESSION_DRIVER	database	Sessions persistantes.
CACHE_STORE	database	Cache persistant via PostgreSQL.

Les variables DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME et DB_PASSWORD sont normalement injectées automatiquement par le blueprint depuis `artisanhub-db`.

6. Configurer Fedapay

1. Créer ou ouvrir un compte sur dashboard.fedapay.com.
2. Commencer avec les clés Sandbox.
3. Ajouter dans Render :

```
FEDAPAY_ENV=sandbox

FEDAPAY_PUBLIC_KEY=cle_publique_sandbox

FEDAPAY_SECRET_KEY=cle_secrete_sandbox

FEDAPAY_WEBHOOK_SECRET=secret_du_webhook
```

Tester création de paiement, callback, confirmation, remboursement et reversement. Passer à FEDAPAY_ENV=production seulement après validation du flux Sandbox.

Documentation : docs.fedapay.com.

7. Configurer l'email SMTP

Exemple avec Brevo SMTP :

```
MAIL_MAILER=smtp
```

```
MAIL_HOST=smtp-relay.brevo.com
```

```
MAIL_PORT=587
```

```
MAIL_USERNAME=adresse-email
```

```
MAIL_PASSWORD=cle-smtp
```

```
MAIL_FROM_ADDRESS=no-reply@votre-domaine.com
```

```
MAIL_FROM_NAME=ArtisanHub
```

Vérifier le domaine expéditeur et les enregistrements SPF, DKIM et DMARC. Tester inscription, vérification, support, commande et remboursement.

8. Configurer le stockage S3, R2 ou Spaces

Render ne doit pas être utilisé comme stockage permanent des documents. Créer un bucket privé chez AWS S3, Cloudflare R2, DigitalOcean Spaces ou Wasabi.

```
FILESYSTEM_DISK=s3
```

```
AWS_ACCESS_KEY_ID=...
```

```
AWS_SECRET_ACCESS_KEY=...
```

```
AWS_DEFAULT_REGION=...
```

```
AWS_BUCKET=artisanhub
```

```
AWS_ENDPOINT=
```

```
AWS_URL=
```

```
AWS_USE_PATH_STYLE_ENDPOINT=false
```

Pour Cloudflare R2, utiliser un endpoint de la forme :

```
https://ACCOUNT_ID.r2.cloudflarestorage.com
```

Tester upload et lecture contrôlée d'un document KYC et d'une preuve de livraison. Le bucket doit rester privé.

9. Configurer WhatsApp, SMS et Sentry

```
WHATSAPP_DRIVER=production

WHATSAPP_TOKEN=...

WHATSAPP_PHONE_ID=...

SMS_GATEWAY_URL=...

SMS_GATEWAY_KEY=...


SENTRY_LARAVEL_DSN=https://...

SENTRY_ENVIRONMENT=production

SENTRY_TRACES_SAMPLE_RATE=0.2
```

Si aucun fournisseur réel n'est prêt, garder temporairement `WHATSAPP_DRIVER=log` et ne pas annoncer l'envoi réel aux utilisateurs.

10. Configurer le worker

Dans `artisanhub-worker`, reprendre les variables suivantes :

```
APP_KEY

APP_URL

DB_CONNECTION
```

DB_HOST

DB_PORT

DB_DATABASE

DB_USERNAME

DB_PASSWORD

QUEUE_CONNECTION

MAIL_*

FEDAPAY_*

AWS_*

La commande du worker est :

```
php artisan queue:work --sleep=3 --tries=3 --timeout=90
```

Les logs doivent montrer que le worker démarre sans erreur et traite les jobs.

11. Déploiement et migrations

Le blueprint lance avant le démarrage du web service :

```
php artisan migrate --force
```

Le conteneur met ensuite en cache :

```
php artisan storage:link --force
```

```
php artisan config:cache
```

```
php artisan route:cache
```

```
php artisan view:cache
```

Ne jamais lancer `migrate:fresh` en production.

12. Vérifications après déploiement

```
curl -i https://artisanhub-web.onrender.com/up
```

```
curl -i https://artisanhub-web.onrender.com/health
```

Contrôler dans Render :

- Le health check répond avec succès.
- Les migrations sont terminées.
- Le worker est en état **Live**.
- Le cron s'exécute chaque minute.
- Aucune erreur répétée n'apparaît dans les logs.
- Les fichiers uploadés sont bien envoyés vers S3/R2.

13. Tests fonctionnels minimum

1. Créer un compte client.
2. Créer ou valider un compte artisan.
3. Créer une commande.
4. Tester le paiement FedaPay Sandbox.
5. Accepter et affecter la commande.
6. Tester une livraison et une preuve de livraison.
7. Ouvrir un ticket support.
8. Envoyer un email réel.
9. Tester un job asynchrone.
10. Tester les logs et une erreur contrôlée dans Sentry.

14. Dépannage courant

Problème	Vérification
Application en erreur 500	Vérifier APP_KEY, logs Render, APP_DEBUG=false et connexion DB.
Migrations échouées	Vérifier DB_CONNECTION=pgsql et les variables DB injectées.

Emails non envoyés	Vérifier MAIL_HOST, port 587, identifiants SMTP et domaine expéditeur.
Uploads perdus après redéploiement	Configurer FILESYSTEM_DISK=s3 et un bucket durable.
Jobs bloqués	Lire les logs du worker et exécuter queue:failed dans un shell Render.
Paieement refusé	Vérifier environnement Sandbox, clés, webhook et URL publique.

15. Checklist finale

- **APP_KEY** configurée et non exposée.
 - **APP_DEBUG=false.**
 - PostgreSQL créée et migrations terminées.
 - SMTP testé avec un vrai email.
 - Fedapay Sandbox testé.
 - S3/R2 testé pour les documents.
 - Worker et scheduler actifs.
 - Health check /up fonctionnel.
 - Logs et alertes vérifiés.
 - Backup PostgreSQL et restauration planifiée.

16. Liens utiles

- [Render Blueprint Specification](#)
- [Render Docker Deployment](#)
- [Laravel Deployment](#)
- [Fedapay Documentation](#)
- [Cloudflare R2](#)
- [Brevo API/SMTP](#)
- [Sentry](#)